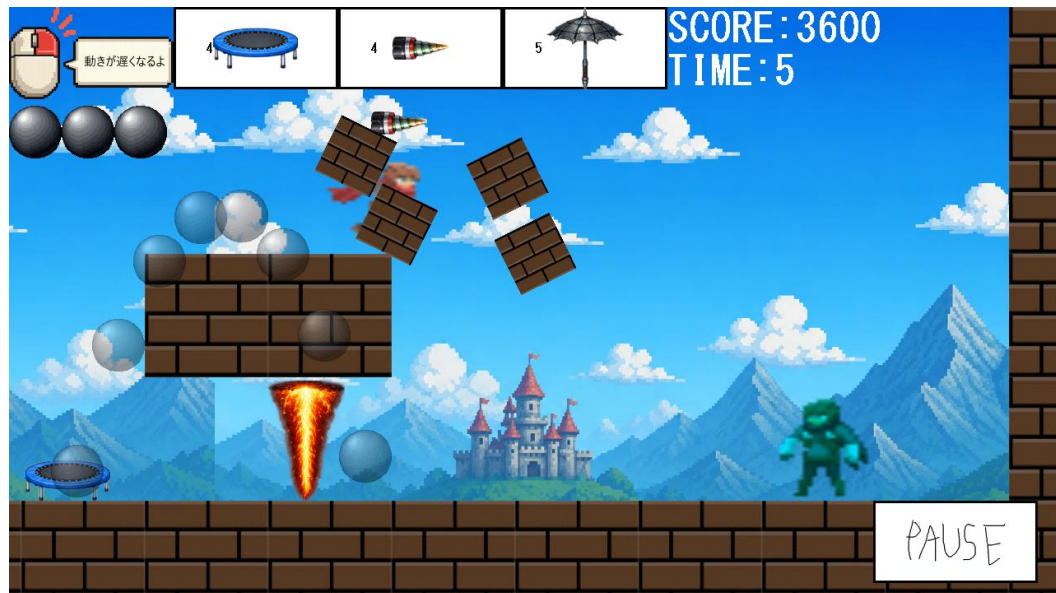


PORTFOLIO

Game Programmer



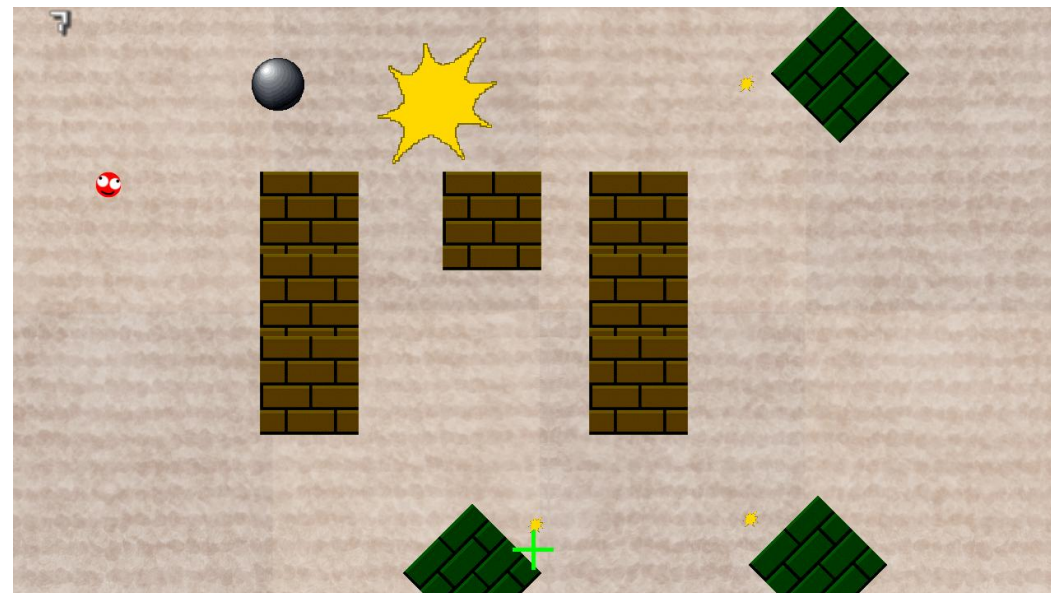
とまれない

【2Dアクションゲーム】

使用技術

C++

Visual Studio



跳弾

【2Dシューティングゲーム】

Game Programmer
吉岡 市之介

CONTENTS

01 Profile

02 とまれない
2D横スクロールシミュレーション

03 跳弾
2Dシューティングゲーム

04 学んだこと

05 プレイ映像URL

01 PROFILE

氏名：吉岡 市之介

志望職種：ゲームプログラマー

使用言語

- ・ C++

開発環境

- ・ Visual Studio

得意分野

- ・ ゲームプログラミング
- ・ オブジェクト指向
- ・ 保守性・拡張性を意識した設計

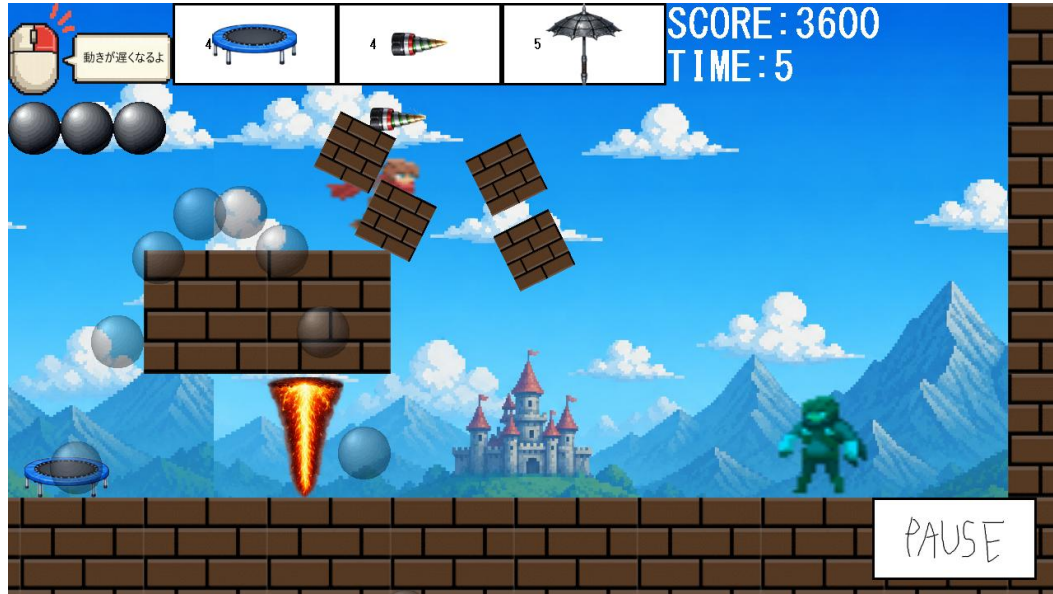
個人制作 2作品

- ・ とまれない
- ・ 跳弾

自己PR

私は、保守性・拡張性を意識しながらゲーム開発に取り組むことを大切にしています。個人制作ではC++を用いて2Dゲームを開発し、プレイヤーや敵AI、当たり判定などの実装を担当しました。仕様変更にも対応しやすい設計を心掛け、クラスの責務を整理し共通処理をまとめるなど改善を重ねました。今後も技術力を磨き、チームで高品質なゲーム開発に貢献できるプログラマーを目指します。

02 とまれない 2D横スクロールシミュレーション



ジャンル

2D横スクロールシミュレーション

制作期間

5ヶ月（週に3回7時間）

開発環境

C++

Visual Studio

制作形態

個人製作

コンセプト

プレイヤーが状況に応じてアイテムを自由に選択し、それぞれの特性を活かしてステージを攻略する。

02 ゲームシステム



■ アイテム選択
プレイヤーが自由に
アイテムを選択設置可能



■ 敵
敵との接触でゲームオーバー

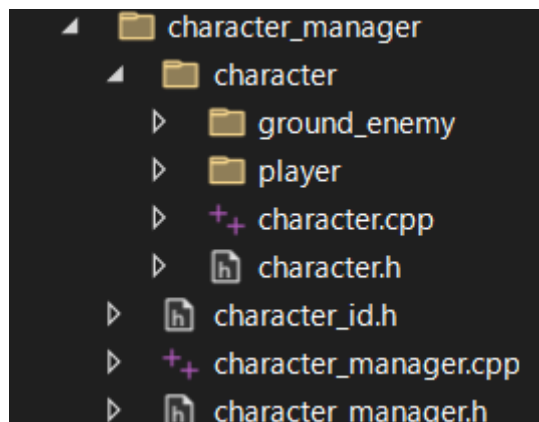


■ ギミック
もともとステージに設置され
ているギミック

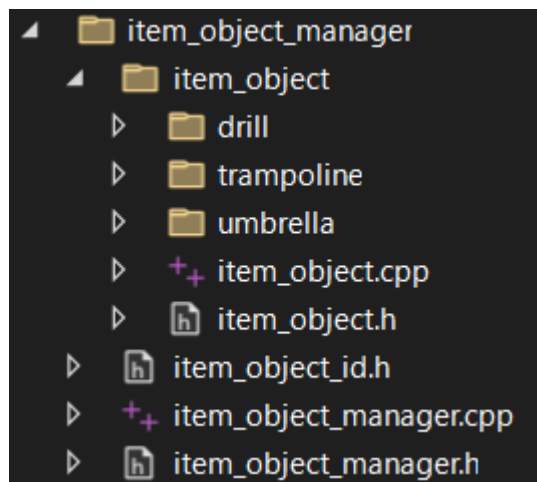


■ クリア条件
ゴールに接触でクリア

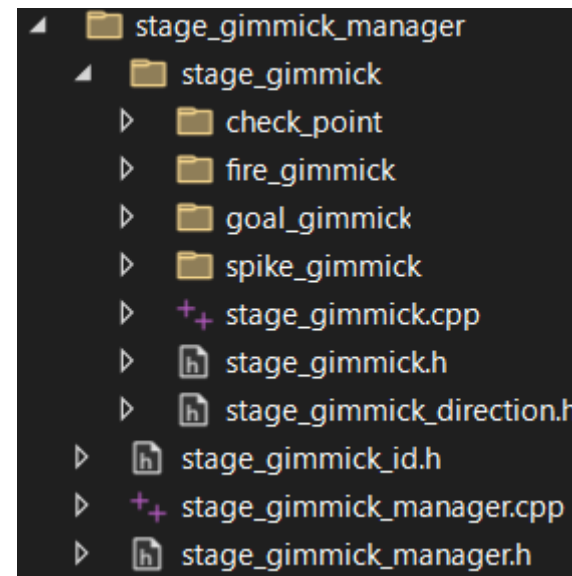
02 プログラム設計・工夫した点



Character
├── Player
└── GroundEnemy



ItemObject
├── Trampoline
├── Drill
└── Umbrella



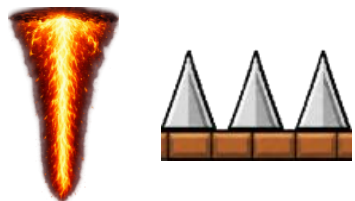
StageGimmick
├── Spike
├── Fire
├── Goal
└── CheckPoint

■設計で工夫した点

- ・役割ごとにManagerを分割し、責務を明確化
- ・新しいアイテムやギミックは派生クラスを追加するだけで拡張できる構成にした

- ・基底クラスを作成し、継承によって共通処理を集約

02 ギミックの実装・工夫した点



接触したキャラに対してダメージ



リスポーン位置更新

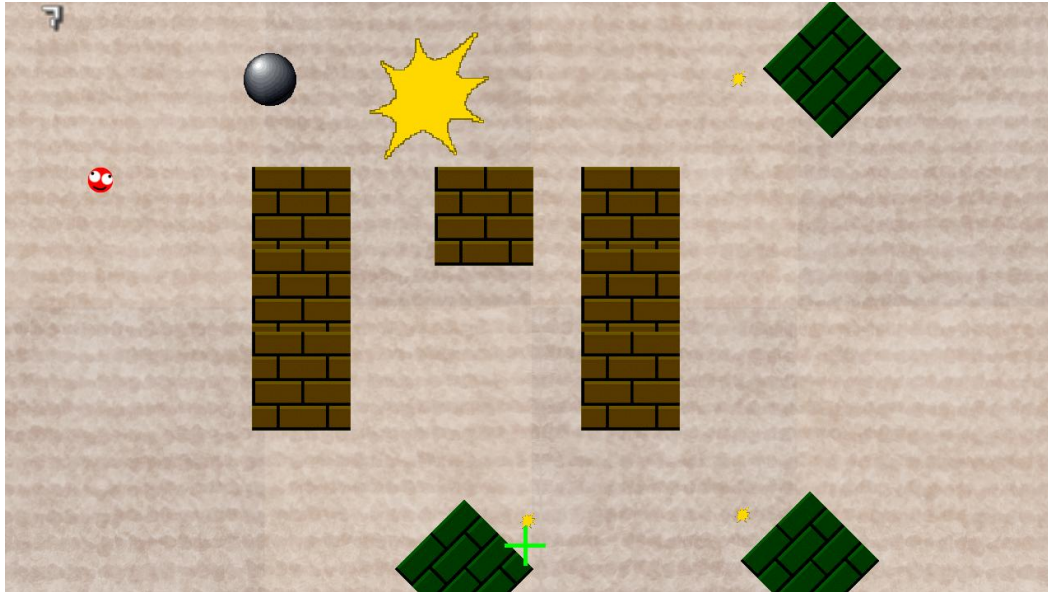


接触するとクリア

工夫した点

- StageGimmickを基底クラスとして設計
- 各ギミックを派生クラスで実装
- 新しいギミックを追加しやすい構成

03 跳弾 2Dシューティングゲーム



ジャンル
2Dシューティング

制作期間
5ヶ月（週に3回7時間）

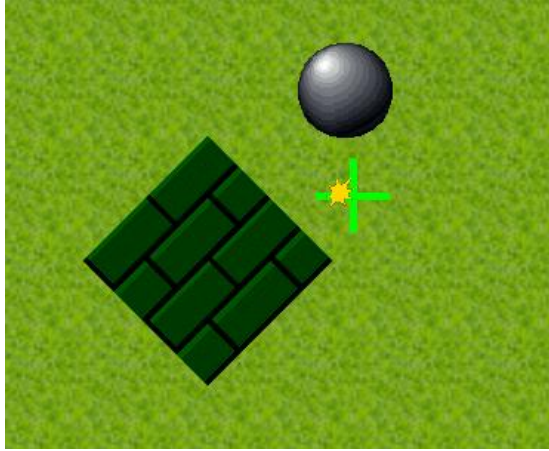
開発環境
C++
Visual Studio

制作形態
個人制作

コンセプト

敵との位置関係や跳弾角度を考えながら攻略する。

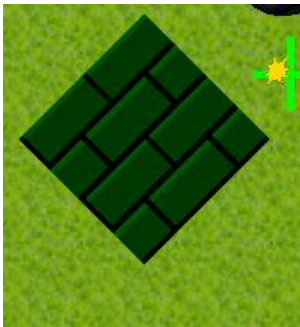
03 ゲームシステム



跳弾
弾が接触した
ブロックや壁
の角度に対し
て反射



敵
弾が命中すると
撃破



ブロック
自分で設置し
て回転できる



クリア条件
敵を全滅させる

03 プログラム設計・工夫した点

跳弾 (弾の跳ね返り) の処理

```
void CBullet::HitMove(IBlock* block)
{
    aqua::CVector2 object_position = block->GetCenterPosition();
    float object_size = block->GetSize();
    float object_angle = block->GetAngle();

    // オブジェクトからボールまでのベクトルを求めて
    // オブジェクト空間にローカライズ
    aqua::CVector2 v = GetCenterPosition() - object_position;

    // オブジェクトに対するベクトルの角度を求める
    float angle = aqua::RadToDeg(atan2(v.y, v.x));
    float rad = aqua::DegToRad(angle);
    aqua::CVector2 vector;
    vector.x = cos(rad);
    vector.y = sin(rad);

    // オブジェクトの角度に応じてベクトルを回転させる
    vector = aqua::CVector2::RotationZ(vector, -aqua::DegToRad(object_angle));
    v = aqua::CVector2::RotationZ(v, -aqua::DegToRad(object_angle));

    // オブジェクトの角度に応じて加速度を回転させる
    aqua::CVector2 localized_velocity = aqua::CVector2::RotationZ(m_Velocity, -aqua::DegToRad(object_angle));

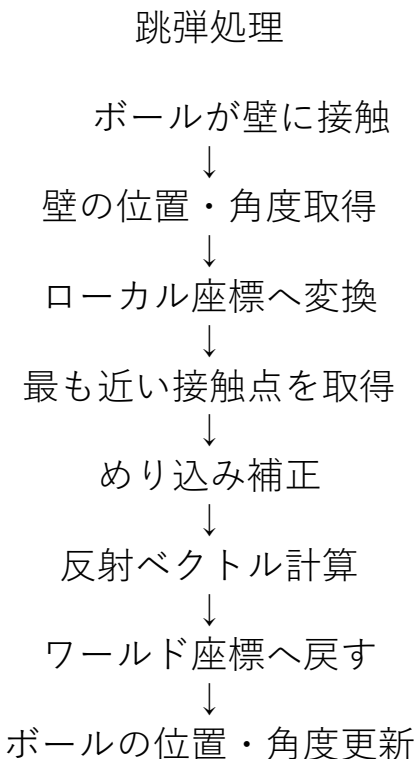
    // オブジェクトの当たり位置を求める
    aqua::CVector2 near_position = GetNearPosition(v, vector, object_angle, object_size);

    // 跳ね返った後のVelocityの更新
    switch (block->GetDirectionLocal(near_position)) { ... }
    // オブジェクトに埋まらない対策
    v = BackPosition(vector) + (m_Radius + 5.0f) * vector;
    // 角度に応じた加速度に変換
    m_Velocity = aqua::CVector2::RotationZ(localized_velocity, aqua::DegToRad(object_angle));
    // 跳ね返った後の位置
    m_Position = object_position + aqua::CVector2::RotationZ(v, aqua::DegToRad(object_angle)) - aqua::CVector2(m_Radius, m_Radius);
    // 跳ね返った後の角度
    m_Angle = aqua::RadToDeg(atan2(m_Velocity.y, m_Velocity.x));
}
```

工夫した点

跳弾がゲームの核となるため、反射処理を関数としてまとめ、複数の壁でも同じ処理を利用できるように設計しました。また、弾・壁・敵の役割を分離し、機能追加や修正がしやすい構成を意識しました。

跳弾の処理フロー



04 学んだこと

【学んだこと】

- ・ 保守性
- ・ 拡張性
- ・ 責務分割
- ・ 継承

【今後挑戦したいこと】

- ・ 3Dゲーム開発
- ・ 設計力向上
- ・ AI

ゲーム制作を通して、保守性・拡張性を考慮した設計の重要性を学びました。今後も新しい技術を積極的に学び、より高品質なゲーム開発ができるプログラマーを目指します。

05 プレイ映像URL

ありがとうございました。
プレイ動画はこちらからご覧いただけます。

とまれない(約50秒)

https://youtu.be/fFixunml_2Q



跳弾(約1分)

<https://youtu.be/5ANTCj-mXgc>

